SOC L1 LAB · INC-004

Detection, Mitigation, and Triage Guide — INC-004

Persistence is a change-management problem with a file path

INCIDENT	INC-004 — Cron and systemd persistence
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	Critical

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Detection model
2. Rule-by-rule
3. Tuning
4. Triage playbook
5. Severity matrix
6. Containment and mitigation
7. Escalation template
8. Post-incident
9. Common mistakes
10. Coverage map
11. Metrics

Audience: an L1 analyst who receives the alert, and an L2 or detection engineer who owns the rule.

This guide uses INC-004 as the worked example. Rule IDs are Wazuh 4.14.7 stock unless marked custom.

---

1. Detection model

Root persistence produces four signals:

Layer	Source	Question	INC-004 evidence
File	FIM on `/etc`	Did a cron or unit file appear?	100129, 100130
Service manager	systemd	Was it enabled?	`systemctl enable` in audit
Process	auditd execve	What wrote it, and what ran?	80792 `tee`, `systemctl`, `id`
Identity	sudo	Who had root?	5402

The file event proves the object; the command trail proves the actor; the proof file proves execution. None of them proves authorization.

1.1 Signal hierarchy

```
Low     554/550  a file in /etc changed
        5402     someone used sudo
        ----  common; not actionable alone

Medium  100130   /etc/systemd/system changed
        ----  could be a legitimate enable or a new unit

High    100129   /etc/cron.d file added or modified
        100130 + a new .service/.timer body
        ----  persistence candidate; check change control

Critical root execution from the new job/unit
        ----  confirmed persistence
```

---

2. Rule-by-rule

2.1 Rule 100129 — cron file added or modified

```xml
<rule id="100129" level="12">
  <if_sid>550,554</if_sid>
  <match>/etc/cron.d/</match>
  <description>FIM: cron file added or modified (possible persistence)</description>
  <mitre><id>T1053.003</id></mitre>
```

```
  <group>syscheck,persistence,</group>
</rule>
```

- **Parents:** 550 (modified) and 554 (added).
- **Match:** the `/etc/cron.d/` path substring in the FIM log.
- **Tested:** 23:17:25.115 on `/etc/cron.d/updates`.
- **False positives:** any legitimate cron job. The baseline run produced two hits for one legitimate file (add + modify).
- **Gap:** user crontabs in `/var/spool/cron/crontabs` are not covered.

2.2 Rule 100130 — systemd unit or enable-link changed

```
<rule id="100130" level="10">
  <if_sid>550,554</if_sid>
  <match>/etc/systemd/system/</match>
  <description>FIM: systemd unit or enable-link changed (possible persistence)</description>
  <mitre><id>T1543.002</id></mitre>
  <group>syscheck,persistence,</group>
</rule>
```

- **Parents:** 550 and 554.
- **Match:** the `/etc/systemd/system/` path substring.
- **Tested:** 23:17:28.122 (`updates.service`), 23:17:28.128 (`updates.timer`), and 23:17:31.309 (`timers.target.wants/updates.timer`).
- **False positive:** every `systemctl enable`, because it creates a symlink under `*.wants/`. The baseline produced one hit for enabling cron.
- **Why level 10:** enable events are frequent; level 12 would flood the queue. The new `.service`/`.timer` body is the higher-signal part, but the rule cannot distinguish a symlink from a file with one `<match>`.

2.3 Supporting stock rules

Rule	Use
550/554	The underlying FIM event
80790	Audit created a file
80792	`tee`, `systemctl`, `id` command trail
5402	Who ran the sudo command
553	Deletion — not covered by 100129/100130

---

3. Tuning

3.1 Route to change control

Both rules report **change**. The first triage question is "was this approved?", not "is this malicious?". Without a change-ticket lookup, every legitimate cron job and every `systemctl enable` becomes an incident.

3.2 Reduce 100130 noise

Options:

- allow-list approved unit names and known enable symlinks;
- split the rule: `.service` / `.timer` files at level 12, `*.wants/` links at level 5;
- monitor `systemctl enable` from audit and correlate it with the unit write.

A split is preferable to suppression. The enable link alone is weak; the unit body plus the enable plus execution is strong.

3.3 Cover user crontabs

Add FIM and audit coverage for `/var/spool/cron/crontabs` and the `crontab` command. A user-level cron job is a persistence path that `/etc/cron.d` does not see.

3.4 Cover deletion

100129/100130 use 550/554. A deletion is 553. If an attacker removes a unit or cron file, neither rule fires. Add deletion rules for the same paths, at least at level 8.

3.5 Correlate write → enable → execute

A sequence rule or a small enrichment can connect:

1. a new unit body (100130);
2. `systemctl enable` (80792);
3. the service process (80792, `id` /payload);
4. an outbound connection.

The four together are a confirmed persistence event. Individually they are noise.

---

4. Triage playbook

4.1 Five questions

1. **What changed?** Cron file, unit file, or enable symlink?
2. **Who wrote it?** `auid` , `uid` , sudo command, source.
3. **Was it approved?** Change ticket, config-management run, package install.
4. **Did it run?** Proof file, journal, service status, process trail.

5. **What else is there?** Other units, timers, cron files, user crontabs.

4.2 Decision tree

```
Cron/systemd change
│
├─ Package install or approved config-management run
│    └─ Close with the change ID; verify the file hash
│
└─ No approval
    ├─ New cron file or unit body            → High; contain
    ├─ Unit enabled and service executed     → Critical; contain
    ├─ Root execution, outbound connection   → Critical; scope the host
    └─ Deletion only                         → check for anti-forensics
```

4.3 Step by step

1. **Read the FIM path and event.** Added or modified?
2. **Pull the file contents and hash** before deleting.
3. **Check the command trail:** `tee`, `cp`, `systemctl`, `crontab`.
4. **Check whether it ran:** proof file, journal, `systemctl status`, process list.
5. **Check for siblings:** other units, timers, cron files, user crontabs.
6. **Assign severity** with §5.
7. **Contain:** disable the timer, remove the unit/cron file, `daemon-reload`.
8. **Escalate** with the template in §7.

---

5. Severity matrix

Situation	No execution	Executed as user	Executed as root
Cron file added	Medium	High	**Critical**
Unit body added, not enabled	High	High	High
Unit enabled and running	High	High	**Critical**
Enable symlink only	Low	—	—
Approved change	Close	—	—

INC-004: cron and systemd both executed as root → **Critical**.

---

6. Containment and mitigation

6.1 During the incident

Action	Command	Caution
Capture the units	`systemctl cat updates.timer`	Before deletion
Capture the cron file	`cat /etc/cron.d/updates`	Before deletion
Hash the files	`sha256sum ...`	Evidence
Stop the timer	`systemctl disable --now updates.timer`	Stops future runs
Remove units	`rm /etc/systemd/system/updates.*`	Then `daemon-reload`
Remove cron	`rm /etc/cron.d/updates`	Check other names
Check user cron	`crontab -l -u <user>`	`/var/spool/cron/crontabs`
Check other units	`systemctl list-unit-files`, `list-timers`	Compare to baseline

6.2 Hardening

- Configuration management owns `/etc/systemd/system` and `/etc/cron.d`; manual writes are reverted and alerted.
- Least-privilege sudo; time-bound elevation for admin work.
- Allow-list of approved units, timers, and cron jobs.
- Monitor deletions as well as additions.
- Restrict who can run `systemctl enable` and write to `/etc`.

6.3 Detection controls

Control	Purpose
100129	Cron drop-in change
100130	Unit/enable change
80792	Command trail
5402	Actor
553 rule	Deletion/anti-forensics
`/var/spool/cron` watch	User crontabs
Change-ticket enrichment	Authorization

7. Escalation template

```
INC-004 — Cron and systemd persistence
```

```
What:  At 23:17:24 UTC, analyst wrote /etc/cron.d/updates and
       /etc/systemd/system/updates.{service,timer}, then enabled the timer.
       Both execute as root. Proof files contain uid=0(root).
Hashes: cron 991523..., service 1674fa..., timer b58a06...
Detect: 100129 at 23:17:25, 100130 at 23:17:28/31, 80792 tee/systemctl, 5402 sudo.
Done:  Timer disabled, units and cron file removed, daemon-reload, evidence preserved.
Ask:   (1) Was any change approved? (2) Review sudo access for analyst.
       (3) Check other hosts for the same file names/hashes.
```

---

8. Post-incident

- Add the three hashes to a blocklist/watch.
- Add deletion rules for the same paths.
- Add `/var/spool/cron/crontabs` coverage.
- Allow-list approved units; split 100130 into body vs enable-link severity.
- Test the full chain (write → enable → execute) in detection CI.
- Review sudo policy for the actor.

---

9. Common mistakes

Mistake	Consequence
Suppressing 100130 because enable is noisy	Miss the new unit body
Closing on "it's just a cron job"	It runs as root every minute
Deleting the unit before hashing	Evidence lost
Forgetting `systemctl daemon-reload`	The unit may remain active
Ignoring `/var/spool/cron/crontabs`	User persistence survives
Treating the enable symlink as an incident alone	Alert fatigue
Not checking whether the job ran	File presence is not execution

---

10. Coverage map

Technique	ID	Covered by	Gap
Cron	T1053.003	100129	User crontab; deletion
systemd service	T1543.002	100130	`/lib/systemd/system` ; symlink noise
Sudo	T1548.003	5402	Authorized admin use
Root execution	—	80792 + proof	No automatic sequence rule
Persistence removal	—	553 only	Not covered by 100129/100130

11. Metrics

Metric	INC-004
File write → 100129	<1 s
Unit write → 100130	<1 s
Enable → 100130 symlink	<1 s
Legit baseline	2× 100129, 1× 100130
Attack alerts	1× 100129, 3× 100130
Execution	root (uid 0)
Time to contain	minutes (exercise)

The rules detected every file change. The operational work is deciding which changes were authorized.